Hyrje në algoritme dhe programim

Alion Emini

kurseinformatike.com

Përmbajtja

1. Hyrje në algoritme
2. Pseudokodi
3. Flowchart
4. Simbolet kryesore të flowchart
5. Ushtrime/Detyra
6. Përmbledhje

1 Hyrje në algoritme

2 Pseudokodi

3 Flowchart

4 Simbolet kryesore të flowchart

5 Ushtrime/Detyra

6 Përmbledhje

Pse fillojmë me algoritme, jo me kod?

Që të programosh mirë:

- më e rëndësishme se sintaksa: **si mendon për problemin**;

Pse fillojmë me algoritme, jo me kod?

Që të programosh mirë:

- më e rëndësishme se sintaksa: **si mendon për problemin**;
- programimi i mirë fillon gjithmonë me një algoritëm të qartë;

Pse fillojmë me algoritme, jo me kod?

Që të programosh mirë:

- më e rëndësishme se sintaksa: **si mendon për problemin**;
- programimi i mirë fillon gjithmonë me një algoritëm të qartë;
- algoritmi vjen *përpara* çdo gjuhe programimi.

Pyetje për ty!

Mendo:

- Si do ta shpjegoje vetë: "Çfarë është një algoritëm?";

Pyetje për ty!

Mendo:

- Si do ta shpjegoje vetë: "Çfarë është një algoritëm?";
- Kur dëgjon "hap pas hapi", çfarë të vjen në mendje;

Pyetje për ty!

Mendo:

- Si do ta shpjegoje vetë: "Çfarë është një algoritëm?";
- Kur dëgjon "hap pas hapi", çfarë të vjen në mendje;
- Jep një shembull nga jeta e përditshme me hapa të renditur;

Pyetje për ty!

Mendo:

- Si do ta shpjegoje vetë: "Çfarë është një algoritëm?";
- Kur dëgjon "hap pas hapi", çfarë të vjen në mendje;
- Jep një shembull nga jeta e përditshme me hapa të renditur;
- A ekzistojnë algoritmet vetëm në kompjuter?

Ide kryesore

Nga diskutimi duam të themi që:

- algoritmi = **hapa të renditur**;

Ide kryesore

Nga diskutimi duam të themi që:

- algoritmi = **hapa të renditur**;
- hapat duhet të jenë **të qartë** dhe **të saktë**;

Ide kryesore

Nga diskutimi duam të themi që:

- algoritmi = **hapa të renditur**;
- hapat duhet të jenë **të qartë** dhe **të saktë**;
- çojnë në një **qëllim të saktë** (zgjidhje problemi).

Përkufizimi i algoritmit

Algoritmi

Algoritëm quajmë një **bashkësi të fundme** hapash **të qartë**, **të saktë** dhe **të renditur**, që përshkruajnë mënyrën si zgjidhet një problem ose kryhet një detyrë. Duke nisur nga input-i i vlefshëm, ekzekutimi i hapave prodhon gjithmonë output të parashikueshëm dhe korrekt.

Algoritmi dhe programi

- Algoritmi:

Algoritmi dhe programi

- Algoritmi:
  - **recetë logjike** që tregon çfarë duhet bërë;

Algoritmi dhe programi

- Algoritmi:
  - **recetë logjike** që tregon çfarë duhet bërë;
  - **plani i zgjidhjes**, para çdo gjuhe programimi.

Algoritmi dhe programi

- Algoritmi:
  - **recetë logjike** që tregon çfarë duhet bërë;
  - **plani i zgjidhjes**, para çdo gjuhe programimi.
- Programi:

Algoritmi dhe programi

- Algoritmi:
  - **recetë logjike** që tregon çfarë duhet bërë;
  - **plani i zgjidhjes**, para çdo gjuhe programimi.
- Programi:
  - zbatimi teknik i algoritmit në Python, C, Java, etj.;

Algoritmi dhe programi

- Algoritmi:
  - **recetë logjike** që tregon çfarë duhet bërë;
  - **plani i zgjidhjes**, para çdo gjuhe programimi.
- Programi:
  - zbatimi teknik i algoritmit në Python, C, Java, etj.;
  - nëse algoritmi është i gabuar/paqartë, edhe kodi do të jetë.

Pse algoritmet janë thelbësore

Pse janë kritike në programim

- Fillestarët që nisin direkt me kod:

Pse algoritmet janë thelbësore

Pse janë kritike në programim

- Fillestarët që nisin direkt me kod:
  - marrin gabime logjike;

Pse algoritmet janë thelbësore

Pse janë kritike në programim

- Fillestarët që nisin direkt me kod:
  - marrin gabime logjike;
  - shkruajnë kod të ngatërruar;

Pse algoritmet janë thelbësore

Pse janë kritike në programim

- Fillestarët që nisin direkt me kod:
  - marrin gabime logjike;
  - shkruajnë kod të ngatërruar;
  - e kanë të vështirë ta ripërdorin apo zgjerojnë.

Pse algoritmet janë thelbësore

Pse janë kritike në programim

- Fillestarët që nisin direkt me kod:
  - marrin gabime logjike;
  - shkruajnë kod të ngatërruar;
  - e kanë të vështirë ta ripërdorin apo zgjerojnë.
- Me algoritëm të qartë:

Pse algoritmet janë thelbësore

Pse janë kritike në programim

- Fillestarët që nisin direkt me kod:
  - marrin gabime logjike;
  - shkruajnë kod të ngatërruar;
  - e kanë të vështirë ta ripërdorin apo zgjerojnë.
- Me algoritëm të qartë:
  - kodi bëhet më i thjeshtë, më korrekt, më i lexueshëm.

Cilat veti duhet të ketë një algoritëm?

Jo çdo listë hapash është algoritëm i mirë.

- Fundi

Cilat veti duhet të ketë një algoritëm?

Jo çdo listë hapash është algoritëm i mirë.

- Fundi
- Renditja e qartë

Cilat veti duhet të ketë një algoritëm?

Jo çdo listë hapash është algoritëm i mirë.

- Fundi
- Renditja e qartë
- Qartësia

Cilat veti duhet të ketë një algoritëm?

Jo çdo listë hapash është algoritëm i mirë.

- Fundi
- Renditja e qartë
- Qartësia
- Ekzekutueshmëria

Cilat veti duhet të ketë një algoritëm?

Jo çdo listë hapash është algoritëm i mirë.

- Fundi
- Renditja e qartë
- Qartësia
- Ekzekutueshmëria
- Korrektësia

Cilat veti duhet të ketë një algoritëm?

Jo çdo listë hapash është algoritëm i mirë.

- Fundi
- Renditja e qartë
- Qartësia
- Ekzekutueshmëria
- Korrektësia
- Input / Output të përcaktuar

1. Fundi

- Duhet të mbarojë pas një numri të kufizuar hapash.

1. Fundi

- Duhet të mbarojë pas një numri të kufizuar hapash.
- Shembull jo i vlefshëm:

1. Fundi

- Duhet të mbarojë pas një numri të kufizuar hapash.
- Shembull jo i vlefshëm:

  "Merri numrat 1,2,3,. . . dhe vazhdo t'i shtosh përgjithmonë."

1. Fundi

- Duhet të mbarojë pas një numri të kufizuar hapash.
- Shembull jo i vlefshëm:

  "Merri numrat 1,2,3,. . . dhe vazhdo t'i shtosh përgjithmonë."

- Pa fund $\Rightarrow$ jo algoritëm i pranueshëm.

2. Renditja e qartë

- Dihet saktë cili është hapi 1,2,3,...

2. Renditja e qartë

- Dihet saktë cili është hapi 1,2,3,...
- Shembull i keq:

2. Renditja e qartë

- Dihet saktë cili është hapi 1,2,3,...
- Shembull i keq:

  "Piq byrekun, më pas ndiz furrën."

2. Renditja e qartë

- Dihet saktë cili është hapi 1,2,3,...
- Shembull i keq:

  "Piq byrekun, më pas ndiz furrën."

- Rendi i gabuar prish procedurën.

3. Qartësia

- Çdo hap interpretohet njësoj nga të gjithë / kompjuteri.

3. Qartësia

- Çdo hap interpretohet njësoj nga të gjithë / kompjuteri.
- Jo algoritmik: *"Hidh pak kripë."*

3. Qartësia

- Çdo hap interpretohet njësoj nga të gjithë / kompjuteri.
- Jo algoritmik: *"Hidh pak kripë."*
- Algoritmik: *"Hidh 5g kripë."* ose *"1 lugë çaji kripë."*

3. Qartësia

- Çdo hap interpretohet njësoj nga të gjithë / kompjuteri.
- Jo algoritmik: *"Hidh pak kripë."*
- Algoritmik: *"Hidh 5g kripë."* ose *"1 lugë çaji kripë."*
- Kompjuteri nuk supozon asgjë.

4. Ekzekutueshmëria

- Hapat duhet të jenë të realizueshëm në praktikë.

4. Ekzekutueshmëria

- Hapat duhet të jenë të realizueshëm në praktikë.
- Për një fëmijë: *"zgjidh ekuacion diferencial"* nuk është ekzekutues.

4. Ekzekutueshmëria

- Hapat duhet të jenë të realizueshëm në praktikë.
- Për një fëmijë: *"zgjidh ekuacion diferencial"* nuk është ekzekutues.
- Në kod: hapat që mund të përkthehet në instruksione reale.

5. Korrektësia

- Për çdo input të vlefshëm: output i saktë.

5. Korrektësia

- Për çdo input të vlefshëm: output i saktë.
- Shembull jo i mirë:

5. Korrektësia

- Për çdo input të vlefshëm: output i saktë.
- Shembull jo i mirë:

  "Mesatarja e tre numrave: shtoji dhe shumëzoi me 3."

5. Korrektësia

- Për çdo input të vlefshëm: output i saktë.
- Shembull jo i mirë:

  "Mesatarja e tre numrave: shtoji dhe shumëzoi me 3."

- Logjika gabim $\Rightarrow$ algoritëm i pasaktë.

6. Input dhe Output të qarta

- Duhet të dihet çfarë merr dhe çfarë kthen.

6. Input dhe Output të qarta

- Duhet të dihet çfarë merr dhe çfarë kthen.
- Shembull: *mesatarja e tre notave*

6. Input dhe Output të qarta

- Duhet të dihet çfarë merr dhe çfarë kthen.
- Shembull: *mesatarja e tre notave*
- Input: n_1, n_2, n_3.

6. Input dhe Output të qarta

- Duhet të dihet çfarë merr dhe çfarë kthen.
- Shembull: *mesatarja e tre notave*
- Input: n_1, n_2, n_3.
- Output: $M = (n_1 + n_2 + n_3)/3$.

Algoritmi vs. Programi

Forma të ndryshme të algoritmit

- Gjuhë natyrore;

Programi: versioni në Python/C/... i këtij algoritmi. Nëse algoritmi është i gabuar, programi trashëgon problemet.

Algoritmi vs. Programi

Forma të ndryshme të algoritmit

- Gjuhë natyrore;
- Pseudokod;

Programi: versioni në Python/C/... i këtij algoritmi. Nëse algoritmi është i gabuar, programi trashëgon problemet.

Algoritmi vs. Programi

Forma të ndryshme të algoritmit

- Gjuhë natyrore;
- Pseudokod;
- Flowchart;

Programi: versioni në Python/C/... i këtij algoritmi. Nëse algoritmi është i gabuar, programi trashëgon problemet.

Algoritmi vs. Programi

Forma të ndryshme të algoritmit

- Gjuhë natyrore;
- Pseudokod;
- Flowchart;
- Tabela vendimmarrjeje, etj.

Programi: versioni në Python/C/... i këtij algoritmi. Nëse algoritmi është i gabuar, programi trashëgon problemet.

Shembulli 1: Receta e gatimit

Input: përbërësit.

```
Mat sasinë e nevojshme te perberesve.
Hidh perberesit ne ene.
Perziej derisa masa te behet homogjene.
Ndez furren ne temperaturen e caktuar.
Hidh masen ne tepsi.
Fute tepsine ne furre dhe piqe per t minuta.
```

Output: ushqimi i përgatitur.

Pse është algoritëm i mirë?

- Ka fillim dhe fund të qartë.

Pse është algoritëm i mirë?

- Ka fillim dhe fund të qartë.
- Hapa të renditur logjikisht.

Pse është algoritëm i mirë?

- Ka fillim dhe fund të qartë.
- Hapa të renditur logjikisht.
- Hapa të ekzekutueshëm.

Pse është algoritëm i mirë?

- Ka fillim dhe fund të qartë.
- Hapa të renditur logjikisht.
- Hapa të ekzekutueshëm.
- Për input të njëjtë dhe ndjekje korrekte: rezultat i parashikueshëm.

Shembulli 2: Loja me zar

Input: pozicionet fillestare, një zar.

```
Per çdo lojtar me radhe:
    Hidh zarin.
    Nese del 6:
        leviz 2 here.
    Perndryshe:
        leviz aq hapa sa numri ne zar.
    Nese lojtari ka arritur fundin:
        loja perfundon, ky lojtar fiton.
    Perndryshe:
        kalon tek lojtari tjeter.
```

Output: fituesi.

Shembulli 3: Kalimi i rrugës

```
Afrohu te trotuari.
Shiko majtas dhe djathtas.
Nese nuk po vjen asnje makine:
    kalo rrugen.
Perndryshe:
    prit disa sekonda.
    kthehu te "Shiko majtas dhe djathtas".
```

- Kusht (Decision), loop, fundi.

Mesazh për studentët

Çfarë të mbash mend

1. Algoritmi është zemra e programit.
2. Duhet të jetë i qartë, i saktë, i fundëm, i ekzekutueshëm.
3. Shumë procedura të përditshme mund të shkruhen si algoritme.
4. Më vonë: këto ide $\rightarrow$ pseudokod, flowchart, Python.

1 Hyrje në algoritme

2 Pseudokodi

3 Flowchart

4 Simbolet kryesore të flowchart

5 Ushtrime/Detyra

6 Përmbledhje

Pse pseudokod?

Deri tani: algoritme me fjali. Hapi tjetër:

- Pseudokodi është në mes:

Pse pseudokod?

Deri tani: algoritme me fjali. Hapi tjetër:

- Pseudokodi është në mes:
  - më i strukturuar se gjuha natyrore;

Pse pseudokod?

Deri tani: algoritme me fjali. Hapi tjetër:

- Pseudokodi është në mes:
  - më i strukturuar se gjuha natyrore;
  - më i thjeshtë se një gjuhë programimi.

Përkufizimi i pseudokodit

Pseudokodi

Mënyrë për të shkruar algoritme me instruksione të thjeshta, të lexueshme nga njeriu, të ngjashme me kodin, por pa u kufizuar nga sintaksa formale. Qëllimi: kuptueshmëri dhe kalim i lehtë në program.

Përfitimet e pseudokodit

- Qartësi: fokus te logjika.

Përfitimet e pseudokodit

- Qartësi: fokus te logjika.
- Afër kodit: përkthehet lehtë në Python.

Përfitimet e pseudokodit

- Qartësi: fokus te logjika.
- Afër kodit: përkthehet lehtë në Python.
- Pa error sintakse.

Përfitimet e pseudokodit

- Qartësi: fokus te logjika.
- Afër kodit: përkthehet lehtë në Python.
- Pa error sintakse.
- Mjet i mirë komunikimi.

Si duhet të jetë pseudokodi?

- Hapa të mirëpërcaktuar (pa pak, ndoshta).

Si duhet të jetë pseudokodi?

- Hapa të mirëpërcaktuar (pa pak, ndoshta).
- Strukturë e qartë: `Nese`, `Perserit`, etj.

Si duhet të jetë pseudokodi?

- Hapa të mirëpërcaktuar (pa pak, ndoshta).
- Strukturë e qartë: `Nese`, `Perserit`, etj.
- I lexueshëm.

Si duhet të jetë pseudokodi?

- Hapa të mirëpërcaktuar (pa pak, ndoshta).
- Strukturë e qartë: `Nese`, `Perserit`, etj.
- I lexueshëm.
- I lehtë për t'u kthyer në kod real.

1. Sekuenca

```
Lexo A
Lexo B
Shuma = A + B
Shfaq Shuma
```

Instruksionet ekzekutohen njëri pas tjetrit.

2. Leximi i input-it

```
Lexo A
Lexo B
```

3. Shfaqja e output-it

```
Shfaq Shuma
Shfaq "Rezultati eshte:", Shuma
```

4. Veprime aritmetike

```
Shuma = A + B
Mesatarja = (A + B + C) / 3
Diferenca = A - B
```

5. Kushtet (Selection)

```
Nese A > B atehere
    Shfaq "A eshte me i madh se B"
Perndryshe
    Shfaq "A nuk eshte me i madh se B"
FundNese
```

6. Ciklet (Repetition)

```
Per i nga 1 deri ne 5 bej
    Lexo numer
    Shfaq numer
FundPer
```

```
Perserit
    Lexo numer
derisa numer == 0
```

Tre strukturat bazë

Rëndësia

1. Sekuenca

Çdo algoritëm ndërtohet nga këto blloqe.

Tre strukturat bazë

Rëndësia

1. Sekuenca
2. Zgjedhja (Selection)

Çdo algoritëm ndërtohet nga këto blloqe.

Tre strukturat bazë

Rëndësia

1. Sekuenca
2. Zgjedhja (Selection)
3. Përsëritja (Repetition)

Çdo algoritëm ndërtohet nga këto blloqe.

Shembull: Shuma e dy numrave

```
Start
Lexo numrin e pare dhe ruaje ne A
Lexo numrin e dyte dhe ruaje ne B
Shuma = A + B
Shfaq "Shuma e dy numrave eshte:", Shuma
End
```

Nga pseudokodi në program

Shënim

Ky pseudokod kthehet pothuajse rresht më rresht në Python. Pseudokodi ju lejon të testoni logjikën përpara se të luftoni me sintaksën.

1 Hyrje në algoritme

2 Pseudokodi

3 Flowchart

4 Simbolet kryesore të flowchart

5 Ushtrime/Detyra

6 Përmbledhje

Çfarë është një flowchart?

Flowchart

Diagram grafik i hapave të algoritmit, me simbole standarde (Start/End, Process, Input/Output, Decision, Connector) dhe shigjeta që tregojnë rrjedhën e ekzekutimit.

Pse përdorim flowchart-e?

- Bëjnë rrjedhën vizuale dhe të lehtë për t'u ndjekur.

Pse përdorim flowchart-e?

- Bëjnë rrjedhën vizuale dhe të lehtë për t'u ndjekur.
- Të dobishme për diskutim në klasë.

Pse përdorim flowchart-e?

- Bëjnë rrjedhën vizuale dhe të lehtë për t'u ndjekur.
- Të dobishme për diskutim në klasë.
- Na ndihmojnë të kontrollojmë degëzimet dhe rastet e harruara.

1 Hyrje në algoritme

2 Pseudokodi

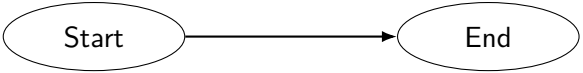
3 Flowchart

4 Simbolet kryesore të flowchart

5 Ushtrime/Detyra

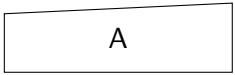
6 Përmbledhje

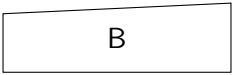
Start / End



Tregon fillimin dhe fundin e algoritmit.

Input





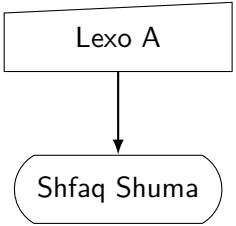
Tregon veprimin e input.

Process

$$\boxed{\text{Shuma} = \text{A} + \text{B}}$$

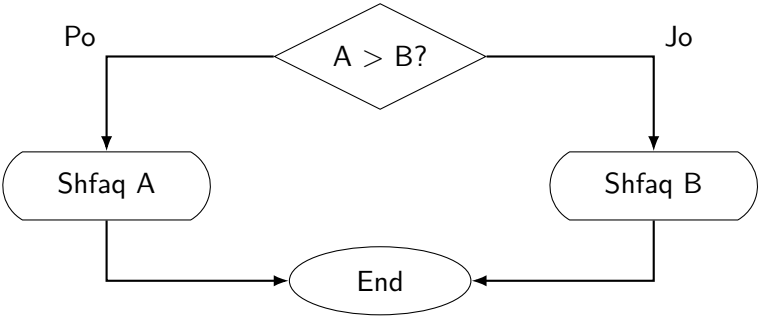
Përpunim / llogaritje.

Input / Output



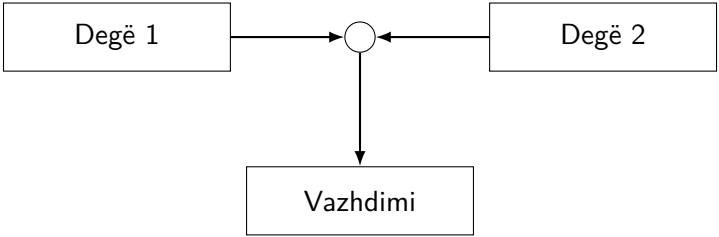
Lexim ose shfaqje e të dhënave.

Decision



Pyetje Po/Jo me dy degë.

Connector



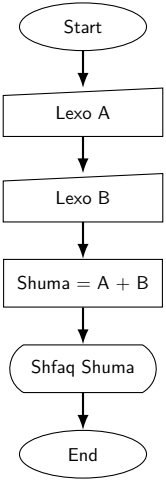
Lidh dy rrjedha në një pikë të përbashkët.

Si të lexosh simbolet

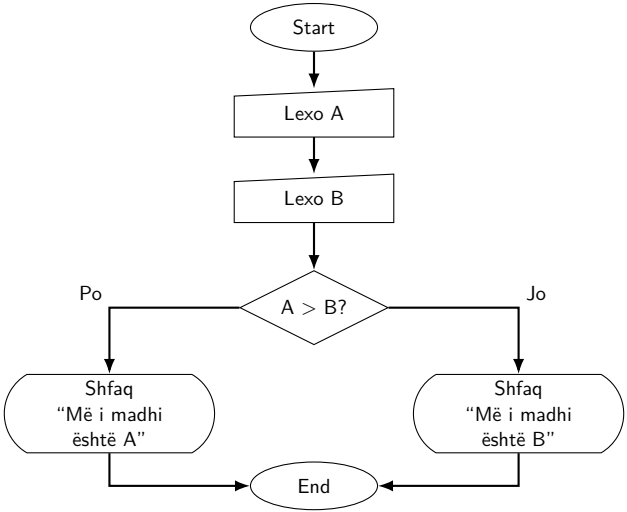
Guidë

- Fillo nga Start dhe ndiq shigjetat.
- Process: çfarë llogaritet?
- Input/Output: çfarë hyn/del?
- Decision: çfarë ndodh për Po dhe për Jo?

Shuma e dy numrave



Gjetja e numrit më të madh



1 Hyrje në algoritme

2 Pseudokodi

3 Flowchart

4 Simbolet kryesore të flowchart

5 Ushtrime/Detyra

6 Përmbledhje

Ushtrimi 1: Sipërfaqja e drejtkëndëshit

Shkruaj një algoritëm që:

- lexon gjatësinë L dhe gjerësinë W të një drejtkëndëshi;
- llogarit sipërfaqen $S = L \times W$;
- shfaq vlerën e sipërfaqes.

Ushtrimi 1: Zgjidhja (Algoritmi & Pseudokodi)

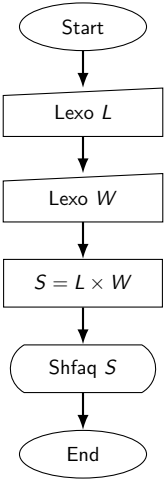
Algoritmi me fjalë:

1. Lexo vlerat e L dhe W.
2. Llogarit $S = L \times W$.
3. Shfaq S.

Pseudokodi:

```
Start
Lexo L
Lexo W
S = L * W
Shfaq "Siperfaqja eshte:", S
End
```

Ushtrimi 1: Zgjidhja (Flowchart)



Ushtrimi 2: A kalon studenti?

Të ndërtohet algoritmi që:

- lexon notën e studentit;
- nëse nota ≥ 50, shfaq `"Kalon"`;
- përndryshe, shfaq `"Nuk kalon"`.

Ushtrimi 2: Zgjidhja (Algoritmi & Pseudokodi)

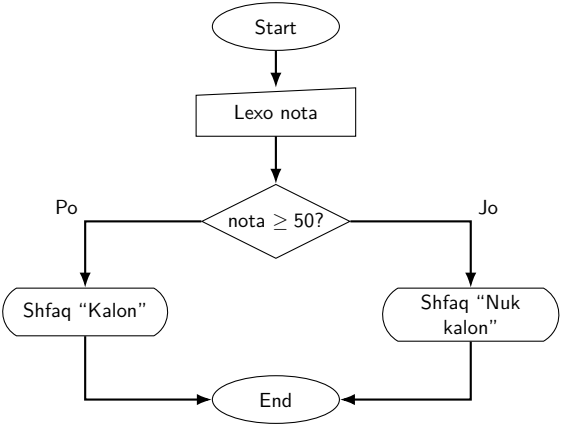
Algoritmi me fjalë:

1. Lexo notën.
2. Nëse nota $\geq$ 50, shfaq "Kalon".
3. Përndryshe, shfaq "Nuk kalon".

Pseudokodi:

```
Start
Lexo nota
Nese nota >= 50 atehere
    Shfaq "Kalon"
Perndryshe
    Shfaq "Nuk kalon"
FundNese
End
```

Ushtrimi 2: Zgjidhja (Flowchart)



Ushtrimi 3: Numri më i madh nga dy vlera

Ndërto algoritmin që:

- lexon dy numra X dhe Y;
- krahaso këto dy vlera;
- shfaq mesazhin `"Më i madhi është X"` ose `"Më i madhi është Y"`.

Ushtrimi 3: Zgjidhja (Algoritmi & Pseudokodi)

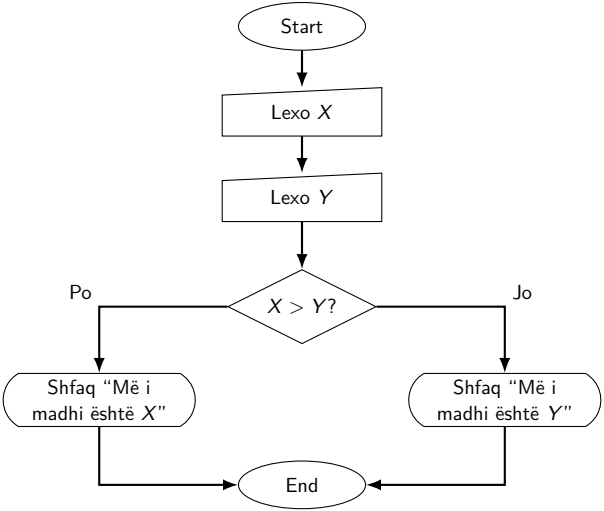
Algoritmi me fjalë:

1. Lexo X dhe Y.
2. Nëse $X > Y$, shfaq "Më i madhi është X".
3. Përndryshe, shfaq "Më i madhi është Y".

Pseudokodi:

```
Start
Lexo X
Lexo Y
Nese X > Y atehere
    Shfaq "Me i madhi eshte X"
Perndryshe
    Shfaq "Me i madhi eshte Y"
FundNese
End
```

Ushtrimi 3: Zgjidhja (Flowchart)



Detyra 1: A është algoritëm i mirë?

Për secilin:

- vendos nëse është algoritëm i vlefshëm;
- shpjego shkurt pse.

```
Hap librin ku te duash dhe lexo derisa te lodhesh.
Merr tre numra, gjej shumen e tyre dhe pjesto me 3. Shfaq
   rezultatin.
Perserit: shto 1 numrit fillestar. (pa kusht ndalimi)
Hidh zarin derisa te dale 6; numero sa here u hodh zar.
```

Detyra 2: Rregullo algoritmin e paqartë

Mbush gotën me pak ujë, shto pak lëng, përziej mirë dhe shërbeje të ftohtë.

- Shpjego pse s'është algoritëm i mirë.
- Jep version të qartë.
- Shkruaj pseudokodin.

Detyra 3: Mesatarja e tre numrave

- Lexo A, B, C; llogarit mesataren; shfaq rezultatin.

Kërkohet:

1. algoritmi me fjali;
2. pseudokodi;
3. flowchart me Start/End, Input, Process, Output.

Detyra 4: Nota (version i zgjeruar)

- nota ≥ 80: Shkëlqyeshëm
- $50 \leq$ nota < 80: Kalon
- nota < 50: Nuk kalon

Kërkohet: pseudokod + flowchart me të tre rastet.

Detyra 5: Numri më i madh nga tre numra

- Lexo X, Y, Z dhe shfaq numrin më të madh.

Kërkohet: pseudokod me Selection + flowchart.

Detyra 6: Temperatura e ujit

Për T (C):

- $T < 0$: Akull
- $0 \leq T \leq 100$: Lëng
- $T > 100$: Avull

Kërkohet: pseudokod + flowchart me degëzime të qarta.

1 Hyrje në algoritme

2 Pseudokodi

3 Flowchart

4 Simbolet kryesore të flowchart

5 Ushtrime/Detyra

6 Përmbledhje

Tre nivelet e përshkrimit

1. **Algoritmi (me fjalë)**: logjika dhe hapat kryesorë.

Tre nivelet e përshkrimit

1. **Algoritmi (me fjalë)**: logjika dhe hapat kryesorë.
2. **Pseudokodi**: urë midis mendimit logjik dhe gjuhës së programimit.

Tre nivelet e përshkrimit

1. **Algoritmi (me fjalë)**: logjika dhe hapat kryesorë.
2. **Pseudokodi**: urë midis mendimit logjik dhe gjuhës së programimit.
3. **Flowchart-i**: pamje grafike e rrjedhës.

Si të punosh me detyrat

1. Mos hidhu direkt te kodi.

Si të punosh me detyrat

1. Mos hidhu direkt te kodi.
2. Së pari shkruaj algoritmin me fjalë.

Si të punosh me detyrat

1. Mos hidhu direkt te kodi.
2. Së pari shkruaj algoritmin me fjalë.
3. Pastaj pseudokodin e strukturuar.

Si të punosh me detyrat

1. Mos hidhu direkt te kodi.
2. Së pari shkruaj algoritmin me fjalë.
3. Pastaj pseudokodin e strukturuar.
4. Vizatimi i flowchart-it kur ka shumë degëzime/cikle.

Si të punosh me detyrat

1. Mos hidhu direkt te kodi.
2. Së pari shkruaj algoritmin me fjalë.
3. Pastaj pseudokodin e strukturuar.
4. Vizatimi i flowchart-it kur ka shumë degëzime/cikle.
5. Në fund: implementimi në Python (ose gjuhë tjetër).

Faleminderit për vëmendjen!

Pyetje?